

2.1 (i) After an IMUL instruction Overflow flag (OF) is clear if lower part of answer is signed extended in upper part, otherwise OF=1

```
mov AL, -4
```

```
mov BL, 4
```

```
imul BL ; AX = FFF0H, OF = 0
```

(ii) • data

```
dividend    sword    -3000
```

```
divisor      sword    25
```

• code

```
mov ax, dividend
```

```
cwd
```

```
idiv divisor
```

(iii)

```
mov ebx, eax
```

$$29 = 32 - 2 = 2^5 - 2$$

```
shl eax, 5 ; *32
```

```
mov ecx, ebx
```

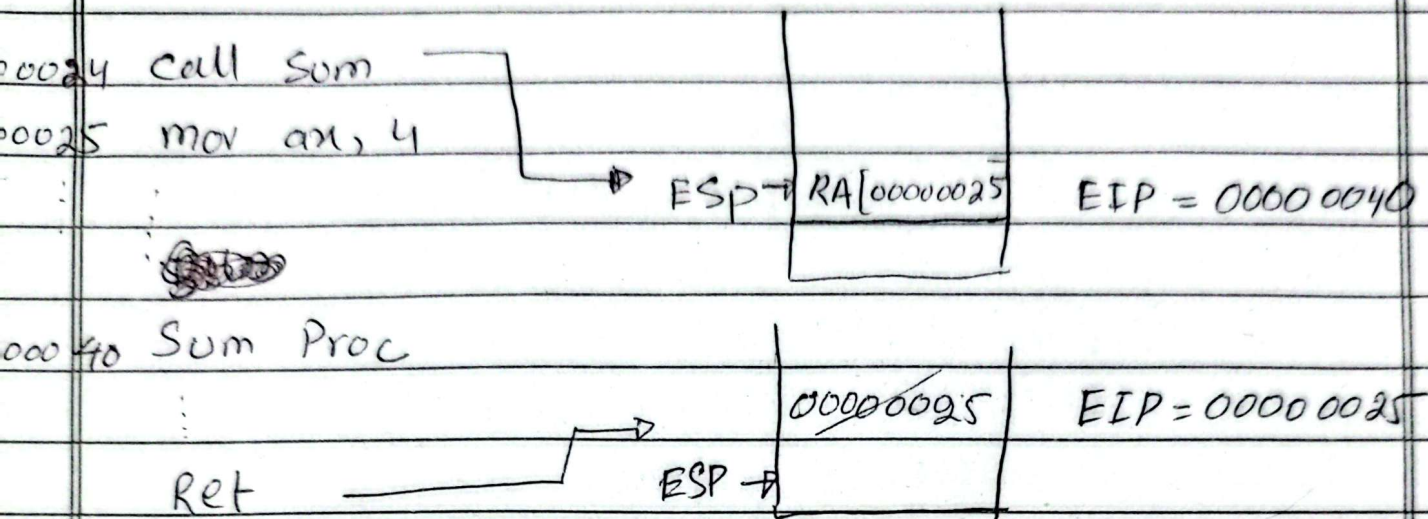
```
shl eax, 1 ; *2
```

```
sub eax, ecx ;
```

```
sub eax, ebx ;
```


(iv) - call instruction $\Rightarrow$ push offset of next instruction on stack & EIP is updated with new offset.

RET instruction $\Rightarrow$ pop the offset of next instruction back into EIP



2a) • data

mark 1 dword 85

mark 2 dword 90

mark 3 dword 80

msg A db "Grade : A", 0

msg B db "Grade : B", 0

msg C db "Grade : C", 0

• code

main Proc

call CalculateGrade

exit

main ENDP

CalculateGrade Proc

call Final Average

cmp eax, 90

jge grade A

cmp eax, 80

jge grade B

jmp grade C

grad A : mov edx, offset msgA
call writestring
jmp done

grad B : mov edx, offset msgB
call writestring
jmp done

grad C : mov edx, offset msgC
call writestring

done :

call Crlf

ret

CalculateGrade ENDP

FindAverage Proc

mov eax, mark1

add eax, mark2

add eax, mark3

mov ebx, 3

cdq ; sign extend for division

idiv ebx

ret

FindAverage ENDP → END main

(b)		
mov cl, 2	cl = 2	
mov al, 7Ch	al = 0111 1100	
mov bl, C7h	bl = 1100 0111	
SHL al, cl ; a = F0h	al = 1111 0000	
SHR bl, cl ; b = 31h	bl = 0011 0001	
inc cl	cl = 3	
SAR bl, cl ; c = 06h	bl = 0001 1000	
ROL al, cl ; d = 87h	al = 0000 1100	
CLC	CF = 0	
DEC cl	cl = 2	
RCL Al, cl ; e = 1Ch	al = 1110 0001	
STC	CF = 1	
RCR Bl, cl ; f = 41h	bl = 0000 1110	
shrd Al, Bl, 2 ; g = 47h	al = 0001 1100	
shrd Bl, Al, 2 ; h = 05h	bl = 000 0011	
	0100 0111	
	4 7	
	Bl (D)	
	0100 0001	
	0000 0101	
	0 5	

Q.3

ESP: 0000 0FF0

• code

main Proc

mov eax, 0000 123A^{al}hmov ebx, 0000 567B^{bl}hmov ecx, 0000 00A0^{cl}hmov edx, 0000 ABCD^{dl}h

add al, 04h ; ax = 123Eh

dec ch ; cx = FFA0h

push eax

push ebx

xchg eax, ecx ; { eax = 0000 FFA0h }

cmp cl, 0h ; { ecx = 0000 123Eh }

Jrz L1 (Not run)

push edx

pop ebx ; ebx = 0000 ABCD^{bl}h

L1:

XOR Bl, 0FFh ; bl = 32h

push ecx

pop eax

main ENDP

END Main

; eax = 0000 123Eh

; ebx = 0000 AB32h

; ecx = 0000 123Eh

; edx = 0000 ABCDh

ESP →

0FF0

0000 123E

0FEC

eax

0000 567B

0FE8

ebx

ESP →

0000 123E

0FE4

ecx

0000 ABCD

edx

0FEC

0FDC

0FD8

Q.4

• data

var1 dword 0

var2 dword 10

var3 dword 6

X dword ?

• code

DO :

Inc var1

mov eax, var1

cmp eax, var3

Jng Else

mov ebx, eax

add ebx, var2

mov x, ebx

Jmp endif

Else :

mov ebx, eax

Sub ebx, var2

mov x, ebx

ENDIF :

cmp eax, var2

Je ENDDO

Jmp DO

ENDDO :